Technical Approach Report: Cynbit AutoBlueprint

Deliverable 3: Implemented Multi-Agent Architecture, Gemini Structured Outputs, Hybrid RAG Pipeline & Executed Verification | Candidate: Sudeep Sagar

1. System Architecture & Dual-Mode Implementation Overview

Cynbit AutoBlueprint is a multi-agent AI discovery and technical blueprint generation platform designed for Cynbit Technologies. The system automates initial project discovery and 18-section architectural blueprinting for customer inquiries submitted via cynbit.in/contact while enforcing the **Multi-Agent Systems (MAS)** and **Human-in-the-Loop (HITL)** governance principles published in How AI Agents Are Transforming Business Operations in 2026.

To ensure full reproducibility across both cloud environments (with live API credentials) and hermetic evaluation sandboxes, every subsystem is implemented with a dual-mode architecture that transparently logs its active execution backend without ever fabricating live LLM or human approval states:

- **Agentic LLM Layer (`services/llm_client.py`)**: Integrates with the Google Gemini API (`gemini-2.5-flash`) via the official `google-genai` SDK, `google-generativeai` SDK, or direct Gemini v1beta REST API using strict JSON schema enforcement and Pydantic validation (`ResearcherLLMOutput`, `ArchitectLLMOutput`, `CriticLLMOutput`). When `GEMINI_API_KEY` is absent or an API error occurs, agents cleanly fall back to a clearly labeled `OFFLINE_DEMO_MODE` deterministic engine.
- **Stateful Graph Orchestration (`orchestration/graph.py`, `orchestration/workflow.py`)**: Uses `langgraph.graph.StateGraph(WorkflowStateTypedDict)` when the `langgraph` package is installed, and provides an API-identical `FallbackStateGraph` engine for offline execution, supporting conditional routing and a bounded Critic-to-Architect revision loop (`max_retries=2`).
- **Semantic & Lexical RAG Pipeline (`rag/`)**: Supports live Gemini semantic embeddings (`text-embedding-004`) and local neural embeddings (`sentence-transformers` `all-MiniLM-L6-v2`), while retaining a deterministic sublinear TF-IDF + domain-synonym + Truncated SVD vector index (`OFFLINE_HYBRID_TFIDF_SVD`) as an automatic fallback.

2. Component-by-Component Implementation Specification

The table below maps each architectural subsystem in the `cynbit-autoblueprint` repository to its live cloud configuration, offline fallback mechanism, and engineering rationale.

Subsystem & Module	Live Cloud Mode (`GEMINI_API_KEY`)	Offline Demonstration Fallback	Verification & Governance
Structured LLM Client (`services/llm_client.py`)	Calls Gemini `gemini-2.5-flash` with `response_mime_type="application/json"` and validates via Pydantic.	Returns `OFFLINE_DEMO_MODE` (or `OFFLINE_FALLBACK_AFTER_API_ERROR`) to trigger rule-guided synthesis.	Tested with mocked Gemini responses, HTTP 503 failures, and malformed JSON payloads.
Multi-Agent Graph (`orchestration/graph.py`)	Compiles `langgraph.graph.StateGraph` over `WorkflowStateTypedDict` with 5 nodes.	Executes `FallbackStateGraph` enforcing a hard ceiling of 15 node transitions.	Prevents infinite agent loops; logs per-node latency and execution mode in SQLite.
RAG Vector Store (`rag/embeddings.py`, `rag/retrieval.py`)	Encodes chunks and queries via Gemini `text-embedding-004` or `all-MiniLM-L6-v2`.	Computes L2-normalized 85% sublinear TF-IDF + 15% non-negative SVD latent vectors.	Enforces `min_similarity=0.08`; unmatched queries report no verified reference found.
HITL Approval & DB (`agents/approval.py`, `database/db.py`)	Stages blueprints as `PENDING_APPROVAL` in SQLite; exports 18-section PDF via ReportLab.	Seeds 4 synthetic inquiries reviewed only by `Synthetic Demo Reviewer A/B`.	Never auto-approves or attributes synthetic demo approvals to real Cynbit personnel.

3. Grounded RAG Corpus & Prompt Engineering Guardrails

To prevent hallucinated case studies or fabricated project details, `rag/ingestion.py` indexes **16 strictly verified public records** from cynbit.in: 11 public case-study listings from cynbit.in/case-studies (**WizHealth**, **Pragya Healths**, **Evento**, **Sofiasa**, **GlobalPay**, **EContentPrep**, **Urban Wardrobe**, **Marketing Addict**, **CarePlus Health**, **Wild basil**, and **Cloudy Tales**), 3 proprietary SaaS product pages from cynbit.in/products (**TenantEase**, **Revio**, and **Cynbit Studio / BitElevate** on cynbit.in/bitelevate/paths and cynbit.in/impact), the company-wide production tech stack on cynbit.in/services, and Cynbit's MAS + HITL engineering article on cynbit.in/blog/ai-agents-business-operations. Because cynbit.in/case-studies does not disclose confidential per-project internal stacks, individual case-study

records contain only verbatim public titles, categories, and featured client quotes, while technology recommendations are drawn from cynbit.in/services.

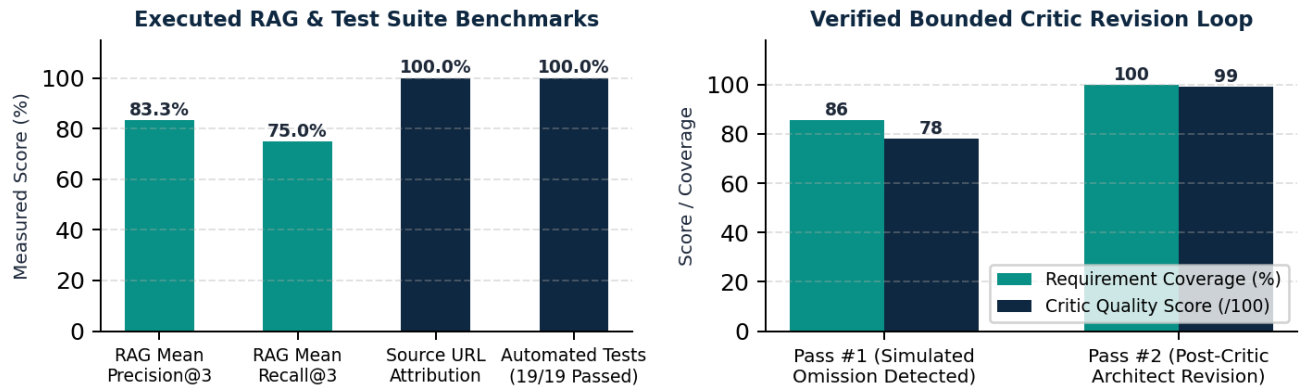


Figure 1: Left — Actual executed RAG benchmark metrics (Precision@3: 83.3%, Recall@3: 75.0%, Source Attribution: 100.0%) and automated test pass rate (19/19). Right — Measured Critic-to-Architect revision loop recovery on inquiry INQ-EVT-103.

Multi-Agent Prompt Engineering & Bounded Revision Workflow

- **Input Validation & Prompt-Injection Firewall (`agents/researcher.py`)**: Validates incoming fields via `ProjectInquiryInput` and neutralizes prompt-injection directives (such as attempts to bypass human approval or expose system prompts) with `[REDACTED_SECURITY_POLICY]`.
- **Agent 1 — Researcher Agent (`prompts/researcher.txt`)**: Classifies the inquiry domain, extracts business objectives, functional/non-functional requirements, constraints, and missing discovery information, and retrieves top-matching verified Cynbit case studies and SaaS products.
- **Agent 2 — Architect Agent (`prompts/architect.txt`)**: Synthesizes all **18 required blueprint sections**, including a Mermaid system architecture diagram, justified technology selections, PostgreSQL/vector database schemas, REST API endpoints, and a 6-stage roadmap. Explicitly labels all effort and timeline figures as *Preliminary Engineering Assumptions*.
- **Agent 3 — Critic Agent (`prompts/critic.txt`)**: Audits requirement coverage, section completeness, security guardrails, preliminary estimate disclaimers, and URL attribution. If defects are found and `revision_count < max_retries`, returns `NEEDS_REVISION` to loop back to the Architect Agent.
- **Agent 4 — Human-in-the-Loop Approval Gate (`agents/approval.py`)**: Holds every blueprint in `PENDING_APPROVAL` status until a human reviewer inspects, optionally edits, and records an explicit `APPROVED`, `REVISION_REQUESTED`, or `REJECTED` decision.

4. Executed Verification, Evaluation Results & Deployment Guide

All metrics reported below are real outputs produced by executing the automated test suite (`python3 -m unittest discover -s tests -t . -v`) and RAG benchmark evaluator (`CynbitVectorRAGIndex().evaluate_benchmark(k=3)`):

Verification Suite	Scope & Test Coverage	Executed Outcome
RAG Pipeline Suite (`tests/test_rag.py`)	HTML cleaning, chunk metadata, unit-norm embeddings, case-study match, unmatched query handling, benchmark metrics.	**6 / 6 Tests PASSED** | Mean Precision@3: **83.33%**, Mean Recall@3: **75.00%**, URL Attribution: **100.0%**.
Agents & Workflow Suites (`tests/test_agents.py`, `tests/test_workflow.py`)	Prompt-injection blocking, requirement extraction, 18-section generation, Critic audit, and bounded revision loop recovery.	**6 / 6 Tests PASSED** | Pass-1 simulated defect (85.7% coverage) automatically recovered to **100.0% coverage** on Pass #2.
HITL & Persistence Suite (`tests/test_approval.py`)	`PENDING_APPROVAL` staging, inline Markdown edits, synthetic reviewer sign-off, SQLite storage, and ReportLab PDF export.	**1 / 1 Integration Test PASSED** | Verified multi-page PDF export (`Sample_Approved_Blueprint.pdf`).
LLM & LangGraph Mocks (`tests/test_llm_integration.py`)	Mocked Gemini structured outputs, HTTP 503 fallback, Pydantic validation error handling, semantic embeddings, LangGraph contract, synthetic reviewer audit.	**6 / 6 Tests PASSED** | Total Automated Suite: **19 / 19 Tests PASSED**.

Local Execution & Live Gemini Configuration

Install dependencies via `pip install -r requirements.txt`. To run in Live Gemini mode, copy `.env.example` to `.env` and set `GEMINI_API_KEY`; otherwise, the application runs automatically in clearly labeled `OFFLINE_DEMO_MODE`. Launch the 6-page Streamlit SaaS application with `streamlit run app.py` or verify CLI seeding and PDF export with `python3 app.py --seed-and-verify`.